

Programming Fundamentals 1 – in-class test

Monday, December 8th 10:15 am

Lecturers: Siobhan Roche, Mairead Meagher

<i>Name</i>	<i>Student Number</i>	<i>Programme(CS/For/Phy) and Group(W1-W4 or Physics)</i>

- ***Instructions:***
- ***This is a 1.5 hour exam.***
- ***3 Questions, answer all.***
- ***Fill in your name, student number and course above***
- ***Complete the areas in this booklet and give it to your invigilator before leaving the room.***
- ***Submit both the skeleton code and the section with your code as a complete booklet.***
- ***You can write anywhere in this booklet and use the back of pages for additional code, rough work, etc.***

For this exam, you are given skeleton code (i.e. fields and method headers) for the classes:

- **Driver**
- **Store**
- **Product**

which make up a Shop System. This is a separate printout.

You will be asked to write code for the following classes:

- **Product**
- **Store**
- **Driver**

Question 1 – Product Class:

Note: you do not have to define the fields or write the getters.

Product.java

*/*Code the the constructor, the setters and the toString() methods as defined in the comment in the following boxes.*

*You should implement the following validation rules :
For the **productName** field, this should be a maximum of 30 characters.
In the case of the constructor, names with more than 30 characters should be cut to the first 20 characters.*

*Hint : You can use the String method **substr** to help you)*

In the case of the setter, do not update the value if it is more than 30 chars long.

*For the **unicCost**, the valid values are between 0 and 500 inclusive) . In the case of the constructor, if an invalid value is input, the default value of 0 should be used.*

In the case of the setter, do not update the value if it is outside the valid values.

*There is no validation needed for the **other two fields**. */*

1.1 Setters :

public void setProductName(String name) { // FILL IN CODE BELOW

}

public void setUnitCost(float unitCost) { // FILL IN CODE BELOW

}

public void setProductCode(int productCode) { // FILL IN CODE BELOW

}

1.2 Constructor:

```
Product(String productName, int productCode, double unitCost, boolean  
inCurrentProductLine)
```

```
{  
//FILL IN THE CODE - VALIDATION RULES SHOULD BE IMPLEMENTED
```

```
}
```

1.3 toString()

```
public String toString() { // FILL IN CODE THAT RETURNS A STRING  
//VERSION OF THE OBJECT
```

```
}
```

2. The skeleton code has a Store class. In this class, there is an array of Product objects, defined as follows:

```
private Product[] products;  
private int total = 0; //dual purpose. 1) number of items stored in array,  
//2) next available index location
```

Store.java

```
// Code each method as defined in the comment in the following boxes.
2.1 - add(..)
// If there is space available, add the product object, passed as
// a parameter, to the array.
// parameter is product which is a Product object to be added to
// the array.
// returns the Status of the add; true for success, false for fail.
```

```
public boolean add(Product product) { //FILL IN CODE BELOW
```

}

2.2 `listProducts()`

```
//The return type is String.
```

```
// This method returns a list of the vehicles stored in the array
// list. Each vehicle should be on a new line and should be
// preceded by the index number e.g.
```

```
//      0: Product A  111  234 is in stock
```

```
//      1: Product B  150  234 is not in stock
```

```
// If there are no products stored in the array list, return a
// string that contains "No products".
```

```
public String listProducts() { // FILL IN CODE BELOW
```

}

2.3 listProductsWithUnitCostOver50()

```
// This method builds and returns a String containing all the  
// products in the array  
// whose unit cost is over 50  
  
// returns a String containing all the products in the array whose  
// product number is over 50, or  
// "No products have unit cost over 50 in this product array ",  
// if none in the array.  
// If there are no products (at all) in the array, the returned  
// String contains "No products in the products array"
```

```
public String listProductsWithUnitCostOver50() { FILL IN CODE BELOW
```

```
}
```

2.4 productWithLowestUnitCost()

```
/** This method goes through the array of products and returns the  
product with the lowest unit cost. If no products exist in the array,  
null should be returned.*/
```

```
public Product productWithLowestUnitCost() { //FILL IN CODE BELOW
```

```
}
```

Question 3: Driver:

The skeleton code has a Driver class. In this class, there is an object of the Store class.

```
public class Driver {  
  
    private Scanner input = new Scanner(System.in);  
    private Store store;  
  
    public static void main(String[] args) {  
        new Driver();  
    }  
    public Driver() {  
        runMenu();  
    } ...  
}
```

This class displays the menu of options that the user can choose from:	<pre>Shop Menu ----- 1) List the Products 2) List the current products 3) Display average product unit cost 4) Display cheapest product 5) List products that are more expensive than a given price 6) Update whether InStock 0) Exit\n</pre>
This class uses the ScannerInput class for reading from the console:	

3.1 addProduct()

In the following box, complete the code for the “Add a Product” menu option. The add method that you will be calling in Driver has the following header:

```
private void addProduct ()
```

3.2 printCheapestProduct()

In the following box, complete the code for the “Print cheapest product” menu option. The add method that you will be calling in Driver has the following header:

```
private void printCheapestProduct ()
```

You should use previously completed methods if this helps.

Driver.java

```
private void addProduct () {
// TODO Write the code to read in the data for a
// product and add it to the Store.

}

private void printCheapestProduct () {
/* TODO Write the code to calculate and print out the details for the
product with the lowest unit cost. If no products exist the output
should contain "No products are in the store". Note you may use any
methods you have already written.*/

}
```